

APPENDIX

A. An Extensive List of SQL-ML Co-Optimization Techniques

We summarize (non-approximate) SQL-ML co-optimization techniques into four main categories, O1-O4. We also highlight the disparate data abstraction (focusing on model parameter representation) and computation abstraction across these categories.

O1. Nested Relational Algebra Optimization [12], [14], [24], [48]. The computation abstraction consists of relational operators such as `filter`, `project`, `join`, `crossJoin`, and `aggregate`. Each operator can be customized by an expression consisting of expression operators, such as `=`, `>`, `<`, `^`, `CALLFUNC` (i.e., invoking an arbitrary opaque function for feature processing, AI/ML model inferences, etc.). In the data abstraction, each weight tensor used by the AI/ML models is encapsulated in the functions, opaque to the query optimizer. Example co-optimization techniques in O1 include:

(Notations: A and B are two attributes of the table R , of vector type and float type respectively, and k is a constant.)

R1-1 filter Reorder. It reorders two filters that are customized by user-defined expressions such as $dnnFraudDetect(R.A) = True$ and $R.B > k$, so that the one with smaller selectivity is executed first to reduce the amount of data input to the other.

R1-2 filter Expression Merge and Factorization. Two filters with different boolean expressions on the same dataset, can be merged into one filter with combined expressions in which redundant computations can be removed, e.g., $f(R.A) = True \wedge R.B > k$ and $R.B < k - 1$ (a naive example) can be merged into $false$. Factorization is the reverse process of merging.

R1-3 filter Pushdown/Pullup. A filter can be pushed down through a join to avoid redundant evaluation of the filter's expression over repeated records caused by the join, if the join's output has a high cardinality. Otherwise, the filter should be pulled up to avoid evaluating expressions over records that do not have a match through the join.

R1-4 project Merge and Factorization. Similar to R1-2, two consecutive projects customized by expressions can be merged into one project with optimized expressions. Factorization is the reverse process of merging.

R1-5 project Pushdown/Pullup. Similar to R1-3, project with expressions can be pushed down/pulled up to avoid redundant/unnecessary computations.

O2. Factorized Inference (derived from factorized ML [27]–[29]). The computation abstraction consists of relational operators and a limited set of factorizable ML functions, such as inner product of vector and weight matrix. For data abstraction, parameters of a factorizable ML function are stored as a factorizable object, visible to the query optimizer. Example co-optimization techniques include:

(Notations: $S(Y, X_S, FK)$ and $R(RID, X_R)$ are two tables. X_S and X_R are the feature vectors. RID is the key attribute of R , and FK is a foreign key of S referencing RID . Y is the union of rest columns in S . After joining S and R on $FK = RID$, we can obtain an output table $T(Y, X)$, where $X \equiv [X_S, X_R]$ is the concatenation of the feature vectors.)

R2-1 Linear Algebra Operator Factorization [27]–[29]. Taking inner product for example, for each feature vector $x \in T.X$, we perform an inner product $w^T x$ as part of the inference computation, where w is a weight matrix. Since T has redundant tuples, redundant inner product computation will occur, which can be avoided by factorizing the inner products over $x \in X$ into inner products over the feature vectors $x_R \in X_S$ and $x_R \in X_R$, leveraging $w^T x = w_S^T x_S + w_R^T x_R$, wherein w_S/w_R is the projection of w to the

features from S/R . Pushing $w_S^T x_S$ down to table S and pushing $w_R^T x_R$ down to table R will avoid redundant computations caused by the join.

R2-2 Factorization of Decision Tree Inference [96], [97]. A k -depth decision tree f contains 2^{k-1} leaf nodes and $2^{k-1} - 1$ intermediate nodes. Each intermediate node is associated with a feature in the input $x \in T.X$, and outputs True/False based on the feature's value. Intermediate nodes are divided and pushed down to R or S based on whether their features are from R or S . The detailed factorized inference process is based on the QuickScorer algorithm [96], [97] that encodes each intermediate node into a 2^{k-1} -length bit vector and converts the decision tree inference result into the bitwise AND of the bit vectors of all False nodes (i.e., intermediate tree nodes that return False on the input feature). The bitwise AND computations can be performed on R and S respectively, and then aggregated.

R2-3 Factorization of Distance Computation [12]. If each feature vector $x \in X$ needs to compute a Euclidean distance with a vector of same length called y , i.e., $dist(x, y)$, we can avoid redundant computation on repeated tuples in T by factorizing the distance computation into $dist(x_S, y_S)$ and $dist(x_R, y_R)$, wherein y_S/y_R is the projection of y to the features from S/R , and pushing down them to S and R respectively, leveraging $dist(x, y) = \sqrt{dist^2(x_S, y_S) + dist^2(x_R, y_R)}$.

O3. Tensor-Relational Transformation. The computation abstraction consists of a limited set of tensor-relational operators [30], such as `aggregate`, `join`, `crossJoin`, `filter`, `project`, `rekey`, `tile`, `concat`, all processing relations. In the data abstraction, the model parameters must be transformed into relations (as discussed in R3-1 to R3-3). Example co-optimization techniques include:

(Notations: similar to the notations in O2, $T(Y, X)$ is a relation, where X is a column of feature vectors.)

R3-1 Linear algebra operators transformed to relational operators [30], [50]–[52]. To enable the transformation, a matrix w could be transformed into a relation $R(colId, tile)$, where each tuple represents a vertically partitioned matrix tile indexed by $colId$, with its data flattened in a 1-D array in the $tile$ attribute. Then taking `matMul` between feature vectors in $T.X$ and w as an example, it can be converted into a `crossJoin` followed by `project`. The `crossJoin` pairs each feature vector in T with each tile in R . Then each joined pair of $x \in T.X$ and $t \in R.tile$ is applied with a `project` that computes $x \times t$. An output block has its row index being the feature vector's row index and its column index being the tile's column index. As described in Tensor Relational Algebra (TRA) [30], other linear algebra computations can also be represented in relational algebra.

R3-2 Decision Forest, e.g., XGBoost and lightGBM, to crossJoin, project, and aggregate [19]. A decision forest model can be represented as a relation $DF(TreeId : INT, TreeObj : TreeType)$, where each tuple represents a decision tree indexed by $TreeId$, and stored as a tree object in the $TreeObj$ column. Applying the model to each feature vector $x \in T.X$, could be transformed into relational processing by first performing a `crossJoin` of T and DF . Then a `project` is applied to each joined pair of $x \in T.X$ and $t \in DF$ to run $t.predict(x)$. Furthermore, the prediction results are aggregated to obtain the final prediction.

R3-3 distances_to_centroids to crossJoin and project. Given a set of clusters represented in a relation $R(ClusterId, C)$, where each tuple represents a cluster indexed by $ClusterId$ with centroid c in the C column. Computing the distance of each feature vector $x \in T.X$ with each centroid $c \in R.C$ can be represented as a `cross_join` of T and R , followed by a `project` that computes the distance between $c \in R.C$ and $x \in T.X$.

The O3 transformations scale to AI/ML models that involve large-scale parameters, by blocking the parameters so that each time only a few blocks need to be loaded into the memory (via buffer pool management). However, such transformation increases overheads for AI/ML models with small parameters [98].

Legend		O1					O2					O3					O4				
Not applicable		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
! Partially support		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
⊕ Supported by external extension		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓ Fully supported		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
DB-centric	CactusDB	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	MADLib [24]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	EvaDB [13]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	PySpark-UDF [85]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Factorized Learning [86]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Factorized	F/SQL [26]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	LMFAO [29]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	JoinBoost [49]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Morpheus [27], [28]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	SimSQL [50]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
RA-centric	SystemML/SystemDS [51], [77]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	TRA [30]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	MASQ [36], [87]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	GaussML [12]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Raven [3]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
SQL	PostgresML [14]	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Fig. 11: Comparison of DB-for-AI/ML systems. (We do not consider approximate co-optimization such as SmartLite [16]. SimSQL, SystemML/SystemDS, and TRA are designed for developing AI/ML workloads, lacking direct support for co-optimization of data-oriented queries and AI/ML, and are therefore marked in gray.)

O4. Data-Driven Optimization of AI/ML Models [3]. The computation abstraction of O4 consists of ML functions, each corresponding to a node in a computational graph. The data abstraction is tensor objects, corresponding to each edge in a computational graph. Example optimization techniques in O4 include:

R4-1 AI/ML Operator Fusion and Factorization. Multiple consecutive AI/ML operators can be fused into a monolithic operator to reduce function call overhead. Factorization is the reverse of the fusing process.

R4-2 AI/ML Library Replacement. An AI/ML operator’s implementation can be replaced by different libraries. For example, for `matMul`, there are different implementations for CPU, GPU, sparse matrices, and dense matrices.

R4-3 AI/ML Algorithm Replacement. For example, a `conv2D` operator can be converted into `matMul` through spatial rewriting of the input images and the filters [55]–[57]. The detailed process is explained in several references [55]–[57].

R4-4 Constant Folding. If an operator f always has an input c (determined through data profiling in the database), it can be replaced with the precomputed constant $f(c)$, eliminating the need for runtime evaluation.

In this paper, we argue that it is important to support all optimization techniques from O1-O4 in one database system. First, it will enable different sub-computations to be applied with different data abstractions and co-optimization techniques, maximizing flexibility. Second, we found combining optimization techniques from different categories will unlock new optimization opportunities. For example, applying R4-1 to factorize a high-level ML function into atomic ML functions such as `matMul` will create new opportunities for applying R2-1 and R1-3 to factorize and push-down the `matMul` or applying R3-1 to convert the `matMul` into relational operators.

B. Variation of Running Examples

In addition to the Running Examples provided in Fig. 3, Fig. 4, and Fig. 5, to illustrate more transformation rules, we provide an variation of the running example in Fig. 3 by changing the two tower recommendation model into DNN-

based recommendation model that has two dense layers. In addition, we also added a filter on the movie features, while the filter uses another two-layer DNN model to predict and decide whether the input tuple should be kept or not. Fig. 12 uses this variation to illustrate the flexibility and expressiveness of our three-level IR. In Fig. 12(a), a query is issued to extract, cross join, and concatenate movie features and user features, and then it applies a DNN model (the `recommendDNN` function) to each concatenated [user, movie] feature vector to calculate the corresponding recommendation score. The results are further filtered using another DNN model (the `popularMovieClassifier` function). A default query plan represented in our three-level IR is shown in Fig. 12(b), where the `Project3` is customized by the `recommendDNN` function, and `Filter1` is customized by the `popularMovieClassifier` function. The details of these functions are described by the middle-level IR consisting of expression operators (i.e., blue nodes). Each opaque expression can be further lowered to bottom-level IR consisting of ML functions (i.e., orange nodes). In Fig. 12(c)–(f), we show how our IR supports different co-optimization techniques from O1-O4.

First, pushing down a filter that is customized by a ML function (R1-2 in O1) can be easily performed at the top-level IR, as shown in Fig. 12(c), where `Filter1` is push down to the `movie` table without changes to other levels of IRs.

Next, as shown in Fig. 12(d), two optimization techniques are applied. First, it applies R2-1, which factorizes the `matMul3` from the bottom-level IR of the `denseLayer3` expression into `matMul3-1` that multiplies the co-factorized weight matrix w_{3-1} with the preprocessed movie features, and `matMul3-2` that performs a similar logic to preprocessed user features. As a result, the `Project3` is factorized into `Project3'`, `Project4`, and `Project5`. `Project3'` corresponds to the rest of the FFNN processing (e.g., the rest of `denseLayer3` and `denseLayer4`). `Project4` and `Project5` correspond to the factorized `matMul3-2` and `matMul3-1` respectively. They are atomic ML functions and thus they are not lowered to bottom-level IRs. Second, R1-3 is applied to push down both `Project4` and `Project5` through the `CrossJoin1` to avoid redundant computations.

Then, in Fig. 12(e), R3-1 is applied to convert the `matMul3-1` and `matMul3-2` into relational operators. In this case, the weight matrices w_{3-1} and w_{3-2} are both vertically partitioned into tiles, e.g., $P3_1(colId : INT, tile : t \in \mathbb{R}^{d1 \times m})$ and $P3_2(colId : INT, tile : t \in \mathbb{R}^{d2 \times m})$, where $d1$ and $d2$ are factorized feature dimension sizes, and m represents the first hidden layer size of the model `recommendDNN`. Therefore, `matMul3-1` is converted into `CrossJoin3` that combines $P3_1$ and the output relation of `Project2`, i.e., the preprocessed movie features in the form of $I(movieId : INT, V : x \in \mathbb{R}^{d1})$, where each tuple represents a $d1$ -dimensional movie feature vector indexed by `movieId`, followed by `Project7` that multiplies each joined pair of $t \in P3_1.tile$ and $x \in R.V$. The transformation of `matMul3-2` is similar.

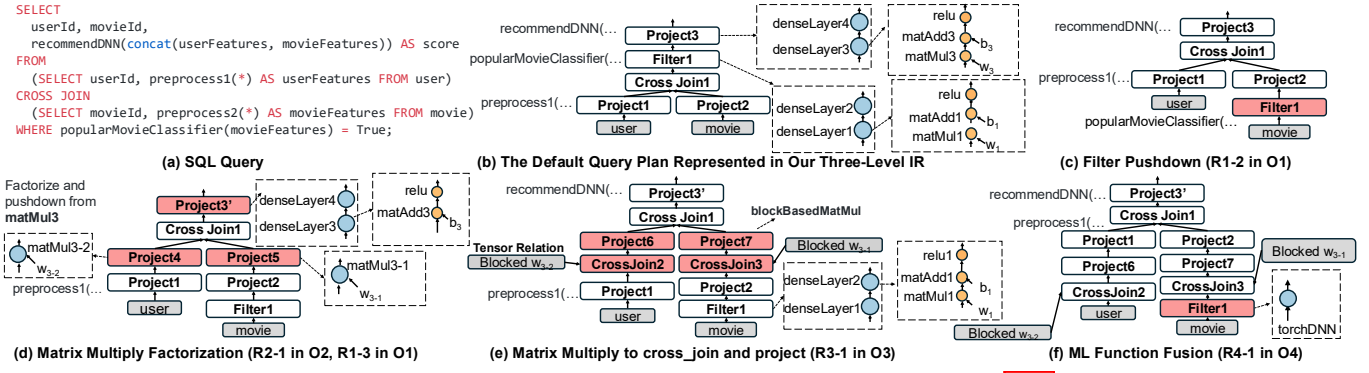


Fig. 12: Examples of our three-level IR. Details are described in Sec. III-E.

Finally, as shown in Fig. 12(f), it applies R4-1 to fuse all operators in the popularMovieClassifier model into one high-level ML function that invokes the torchDNN library.

C. Weisfeiler-Lehman Encoding Process

In this section, we describe how the Weisfeiler-Lehman (WL) kernel is applied to construct positive and negative training samples. The WL kernel encoding procedure is outlined in Algorithm 6. It iteratively generates feature vectors by aggregating and updating node labels. While the encoding processes of Model2Vec and Query2Vec are largely similar, they differ in how the initial node labels are assigned. In Model2Vec, the initial labels are determined by the kernel's FLOPs values. In contrast, Query2Vec assigns initial labels based on different strategies tailored to each node type. These initialization procedures are detailed in Algorithm 7 and Algorithm 9, respectively.

Algorithm 6: Weisfeiler-Lehman Kernel Feature Extraction

```

Input : root: root node of the model tree
         T: number of WL iterations
Output :  $\mathcal{F}$ : WL subtree feature counts
1  $\mathcal{L}, \mathcal{H}, \mathcal{N} \leftarrow \emptyset$ ; // Label map, history, node list
2 DFS(root,  $\mathcal{L}, \mathcal{H}, \mathcal{N}$ ); // Initialize the labels
   through DFS
3 for  $i \leftarrow 1$  to  $T$  do
4    $\mathcal{L}_{new} \leftarrow \emptyset$ ;
5   foreach node  $\in \mathcal{N}$  do
6      $C \leftarrow \text{sorted}([\mathcal{L}[c] \mid c \in \text{node.children}]);$ 
7      $\text{label}_{new} \leftarrow \mathcal{L}[\text{node}] + C$ ;
8      $\mathcal{L}_{new}[\text{node}] \leftarrow \text{label}_{new}$ ;
9      $\mathcal{H}[\text{node}].\text{append}(\text{label}_{new})$ ;
10   $\mathcal{L} \leftarrow \mathcal{L}_{new}$ ;
11  $\mathcal{F} \leftarrow \emptyset$ ; // Feature vector
12 foreach node  $\in \mathcal{N}$  do
13   foreach label  $\in \mathcal{H}[\text{node}]$  do
14      $\mathcal{F}[\text{label}] += 1$ ;
15 return  $\mathcal{F}$ 

```

D. Error Bound Analysis via (p, q) -Consistency

We model query optimization as a discounted MDP $M = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where states are query plans and actions are configurable rewrite-rule instantiations. Let $V^*(s)$ and $Q^*(s, a)$

Algorithm 7: Initial Node Label Assignment for Model2Vec

```

Input : node: a tree node with operator type and FLOPs
          $\mathcal{G}$ : label groups by kernel type
          $\rho$ : FLOPs range threshold
Output: label: unique label assigned to the node
1  $\text{type} \leftarrow \text{node.ml\_op\_type}$ ;
2  $\text{flops} \leftarrow \text{node.ml\_op\_flops}$ ;
3  $\text{group} \leftarrow \mathcal{G}[\text{type}]$ ;
4  $\text{idx} \leftarrow \text{BinarySearchWithRange}(\text{group}, \text{flops}, \rho)$ ;
5 if  $\text{idx} = -1$  then
6    $\text{label} \leftarrow t\_f$ ;
7    $\text{group.append}(\text{flops})$ ;
8    $\text{group.sort}()$ ;
9    $\mathcal{G}[\text{type}] \leftarrow \text{group}$ ;
10 else
11    $\text{label} \leftarrow \text{type} + \text{group}[\text{idx}]$ ;
12 return label

```

Algorithm 8: DFS Traversal for Label Initialization

```

Input : root: root node of the model tree
          $\mathcal{L}$ : node label map
          $\mathcal{H}$ : node label history
          $\mathcal{N}$ : list of all nodes
Output: Updated  $\mathcal{L}$ ,  $\mathcal{H}$ , and  $\mathcal{N}$ 
1 Function DFS(node,  $\mathcal{L}, \mathcal{H}, \mathcal{N}$ ):
2    $\text{label} \leftarrow \text{InitialNodeLabel}(\text{node})$ ;
3    $\mathcal{L}[\text{node}] \leftarrow \text{label}$ ;
4    $\mathcal{H}[\text{node}] \leftarrow [\text{label}]$ ;
5    $\mathcal{N}.\text{append}(\text{node})$ ;
6   foreach child  $\in \text{node.children}$  do
7     DFS(child,  $\mathcal{L}, \mathcal{H}, \mathcal{N}$ );

```

be optimal values, and define the (root) simple regret for the input state s_0 :

$$\mathcal{R}(s_0) = V^*(s_0) - Q^*(s_0, \hat{a}) = \max_{a \in \mathcal{A}(s_0)} Q^*(s_0, a) - Q^*(s_0, \hat{a}), \quad (7)$$

where $\hat{a}$ is the action returned by CACTUSDB after B_{iter} simulations.

The query embedding process $f: \mathcal{S} \rightarrow \mathcal{X}$ is a subjective function mapping ground states $\mathcal{S}$ to their equivalence classes (i.e., abstract states representing clusters of query plans that share the same query embedding) in $\mathcal{X}$. Then, given a logical query plan s_0 , we view the optimization problem as a Markov Decision Process (MDP) $\langle \mathcal{T}, \mathcal{A}, P, R \rangle$ on trajectories, with action space $\mathcal{A}$, state space $\mathcal{T} = \{s_0\} \times (\mathcal{A} \times \mathcal{S})$ being the

Algorithm 9: Initial Node Label Assignment for Query2Vec

Input : node: a tree node with type and model embedding
 $\mathcal{G}$: label groups by node type
 $\mathcal{I}$: FAISS model embedding index
 τ : similarity threshold

Output: label: label assigned to the node

```

1 type ← node.nodeType;
2 group ←  $\mathcal{G}[t]$ ;
3 if node.hashML then
4   q ← normalize(e);
5   idx, sim ←  $\mathcal{I}$ .search(node.mlEmbed);
   // Assign label based on embedding
   // similarity
6   if sim >  $\tau$  then
7     label ← type + idx
8   else
9     label ← type + next_idx;
10     $\mathcal{I}$ .add(node.mlEmbed);
11    next_idx ← next_idx + 1;
// Assign label based on relation data
12 else if type = TableScanNode then
13   label ← t + node.table_id;
// Assign label based on filter predicate
14 else if type = FilterNode then
15   c ← node.filter["colId"];
16   o ← node.filter["opId"];
17   v ← node.filter["val"];
18   label ← t + c + o + v;
// Assign label based on project expression
19 else if type = ProjectNode then
20   label ← t + node.projectExpr;
// Assign label based on join predicate
21 else if type ∈ {HashJoinNode, NestedLoopJoinNode} then
22   label ← t + node.join;
// Assign label based on aggregation keys
23 else if type = AggregationNode then
24   label ← type + node.aggKeys;
25 else
26   label ← type;
27 group.add(label);
28 return label

```

collection of any state-action sequence of any length that starts in s_0 within a length bound d . The transition matrix $\mathcal{P}(t, a, t') = \mathcal{P}(s, a, s')$, which represents that the transition probability of t to t' (one step extension of t) for $a \in \mathcal{A}$ equals to the transition probability of ground state transitioning s to s' for a , where s is the last state in the trajectory t , and s' is t 's one-step extension in t' . We also have $\mathcal{R}(t)$ denoting the reward of selecting s to form trajectory t . The Q function and the value function at trajectory t is defined as usual:

$$Q^*(t, a) = \mathcal{R}(t) + \sum_{t' \in \mathcal{T}} P(t, a, t') V^*(t') \quad (8)$$

$$V^*(t) = \max_{a \in \mathcal{A}} Q^*(t, a) \quad (9)$$

We then define the application of query embedding f to each ground state s in $t \in \mathcal{T}$, such that $f(t) = f(s_0)a_1f(s_1) \dots a_kf(s_k)$. Then, under f , we can define the space of abstracted trajectories at the query embedding level, denoted as $\mathcal{H} = \{h \in \{t_0\} \times (\mathcal{A} \times f) : |h| \leq d\}$. Hence, $\mathcal{H}$

forms partition of $\mathcal{T}$. Then, following existing works on MCTS with state aggregation [6], it is easy to prove the following theorem regarding the error bound of the Q function in our proposed optimization process:

Theorem 2 (reusable MCTS error bound under (p, q) -consistency). Assume our query embedding process f satisfies (p, q) -consistency for $p, q \geq 0$ such that for all $h \in \mathcal{H}$,

$$\exists a^* \in \mathcal{A} : V^*(t) - Q^*(t, a^*) \leq p \quad \forall t \in h \quad (10)$$

$$|V^*(t_1) - V^*(t_2)| \leq q \quad \forall t_1, t_2 \in h. \quad (11)$$

Then, the optimal action in the MCTS tree with state abstraction by f , $a^* = \arg \max_{a \in \mathcal{A}} Q^*(h_0, a)$, has Q^* error in the root state bounded by

$$\left| \max_{a \in \mathcal{A}} Q^*(s_0, a) - Q^*(s_0, a^*) \right| \leq 2d(p + q) \quad (12)$$

E. Proof of Theorem 2

1) *Expectimax-tree abstraction and (p, q) -consistency:* We briefly restate the abstract expectimax-tree construction (specialized to our notation). Let $\mathcal{H}$ be the history partition induced by f , and let $\mu = \{\mu_h\}$ be a family of distributions over ground trajectories $t \in h$. The induced abstract-tree transition is

$$\mathcal{P}_\mu(h, a, h') = \sum_{t \in h} \mu_h(t) \sum_{t' \in h'} \mathcal{P}(t, a, t'),$$

and the abstract value functions satisfy the standard Bellman optimality equations on the history tree:

$$Q_\mu^*(h, a) = \mathcal{R}_\mu(h) + \sum_{h' \in \mathcal{H}} \mathcal{P}_\mu(h, a, h') V_\mu^*(h'), \quad (13)$$

$$V_\mu^*(h) = \max_{a \in \mathcal{A}} Q_\mu^*(h, a), \quad (14)$$

where $\mathcal{R}_\mu(h) = \sum_{t \in h} \mu_h(t) \mathcal{R}(t)$ (see [6]).

We assume (p, q) -consistency, i.e., for each $h \in \mathcal{H}$ there exists a_h^* such that (10)–(11) hold. This matches the definition in [6].

2) *Optimal weighting μ^* and one-step divergence:* In expectimax-tree abstractions, correctness depends not only on f but also on how probability mass is assigned to the ground trajectories inside each history. Hostetler et al. define an *optimal* weighting μ^* and measure deviation from it via a one-step divergence quantity; we reuse that notion here. Let $m \triangleq \max_{h \in \mathcal{H}} \delta(\mu, h) \leq 1$ be the maximum single-step divergence (notation and formal definition as in [6]). When reusable MCTS/UCT aggregates statistics by sampling ground transitions and mapping them to histories, it behaves as if it were using μ^* in the abstract tree (under the conditions discussed in [6]); in the most conservative setting we only assume $m \leq 1$.

3) *Key error decomposition and induction:* Fix any history node h and action a . Define the action-value estimation error

$$E(h, a) \triangleq \left| Q_\mu^*(h, a) - \sum_{t \in h} \mu_h(t) Q^*(t, a) \right|.$$

Following the proof strategy in [6], decompose the error into (i) error propagated from deeper abstract values and (ii) error introduced by aggregating/weighting at the current step:

$$E(h, a) \leq E_Q(h, a) + E_\delta(h, a),$$

where E_Q captures the mismatch between $V_\mu^*(\cdot)$ and the weighted ground optimal values below children, and E_δ captures the single-step weighting/aggregation deviation at the current expansion.

The proof proceeds by induction on the remaining depth k to leaves (base case holds at terminal/absorbing nodes). Assume the inductive hypothesis

$$E(h', a) \leq k(p + mq) \quad \text{for all children } h' \text{ of } h \text{ and all } a \in \mathcal{A}.$$

Using $(p, \cdot)$ -consistency (which allows exchanging max and $\sum$ with additive penalty p) plus the inductive hypothesis yields

$$E_Q(h, a) \leq k(p + mq) + p.$$

This corresponds to the step “exchange max and sum at cost $\leq p$ ” in [6].

For the single-step abstraction error, $(\cdot, q)$ -consistency implies that within any child history the optimal values vary by at most q , and the deviation of μ from μ^* scales this by at most the one-step divergence. This yields

$$E_\delta(h, a) \leq mq,$$

as in the convex-combination argument in [6].

Combining gives the inductive step:

$$E(h, a) \leq (k + 1)(p + mq).$$

Thus at the root history h_0 (depth d), for any action a we have $E(h_0, a) \leq d(p + mq)$.

4) *From value-estimation error to root action suboptimality:* Let $a^\# \in \arg \max_a Q_\mu^*(h_0, a)$ be the abstract-tree optimal action, and let $a^* \in \arg \max_a Q^*(s_0, a)$ be the ground-tree optimal root action. In the worst case, the abstract estimate overvalues $a^\#$ by at most $d(p + mq)$ and undervalues a^* by at most $d(p + mq)$, yielding

$$Q^*(s_0, a^*) - Q^*(s_0, a^\#) \leq 2d(p + mq).$$

This is exactly the final root-step argument in [6]. Since $m \leq 1$, we also obtain the simplified bound $2d(p + q)$. $\square$

F. GPU Acceleration

Further discussion: GPU Support While this work focuses on CPU, our reusable MCTS optimizer is aware of the equivalence between CPU and GPU implementation of the same ML function and leverages R4-2 to invoke LibTorch CUDA library [99] in our GPU environment (see the beginning of Sec. V). As shown in Tab. III, GPU offloading achieved by our optimizer yields an additional 17%–55% performance improvement over the optimized query plan and accelerates model inference by $1.3\times$ on average. However, fully optimizing GPU acceleration raises challenges in pipelining, load balancing, and CPU–GPU

coordination. These aspects are beyond the scope of this paper and are left for future work.

TABLE III: Recommendation Query Performance with GPU Enabled

	EvaDB	Apache PySpark w/UDF	DL-Centric	CactusDB	
				w/o GPU Extension	w/ GPU Extension
Q1	40.40 sec	48.56 sec	47.2 sec	0.14 sec	0.09 sec
Q2	Failed (OOM)	Failed (OOM)	119.54 sec	2.71 sec	2.14 sec
Q3	Failed (OOM)	Failed (OOM)	172.39 sec	18.67 sec	15.93 sec

G. Related Works

In-DB AI/ML Systems. To the best of our knowledge, no existing system fully supports all categories of SQL-ML co-optimization techniques (O1–O4). UDF-centric systems such as EvaDB [13] and PostgresML [14] support most O1 techniques by allowing inference logic to be encapsulated in user-defined functions. MADLib [24] and PySpark [85] provide partial support for O1, but handle arbitrary UDFs poorly. For instance, MADLib cannot push down table-level UDFs like *madlib.mlp_classification*, and PySpark only pushes down MLlib functions and not arbitrary UDFs. These systems cannot analyze or transform UDF internals, making them unsuitable for deeper co-optimization as required by O2–O4. Existing factorized ML systems [26]–[29], [49], [86] focus on ML training processes, relying on predefined factorizable AI/ML models or linear algebra (LA) operators. Although related to O2-style optimization, they lack dynamic decomposition of arbitrary inference pipelines and integration with O3/O4 type optimizations. IMBridge [4], [84] optimizes query performance through automated ML inference context caching and batch inference. Nonetheless, it lacks support for O2 and O3, which are essential for comprehensive end-to-end performance optimization. Other systems like SimSQL [31], [50], SystemML [51], and SystemDS [77] embed ML workloads in relational algebra atop big data processing platforms like Hadoop [100] or Spark [101]. MASQ [36], [87] rewrites limited inference operators (e.g., one-hot encoding, linear models, decision trees) into SQL. None explore partial translation of LA operators into relational form for O2/O4 co-optimization.

Intermediate Representations (IRs). Several unified IRs have been proposed for SQL-ML co-optimization, though most cover only subsets of O1–O4. Raven [3] combines relational, LA, and ML operators, supporting O1 and O4 and partially O3 by converting full ML pipelines into relational form, reducing flexibility. SDQL [54] uses a semi-ring IR covering O1, O2, and O4, but cannot transform LA into relational operators (lacking O3). Weld [32] unifies computation across libraries through vector operators (e.g., *vecbuilder*, *vecmerger*), enabling fusion and data-movement optimization, yet most O1–O3 techniques must be implemented outside of the IR [33]. Lara [53] uses monad comprehensions for UDFs, enabling O1/O4 optimizations (push-down, layout tuning) but not O2/O3. LingoDB [88], [89] leverages MLIR for combining relational and non-relational workloads (e.g., *k*-means, PageRank) [102]. While flexible, it does not target SQL-ML co-optimization techniques in O2–O4. Tensor Relational Algebra [30] primarily addresses O3-style integration of tensors with relational queries, without broader support for O1, O2, or O4.

Inference Query Optimization. Raven [3] applies logical optimizations in a fixed order using rule-based, classification, and regression strategies to choose among ML2SQL, ML2DNN, or no optimization. Most other in-database AI/ML systems rely on rules and heuristics for inference query optimization. Weld [32] and Lara [53] use rule-based plan selection, while SDQL [48] applies rewrite rules until reaching a fixed point.

Database query optimization has evolved into a multifaceted research domain, encompassing rule-based, cost-based, and learning-based approaches. **Rule-based optimizers (RBOs)** apply predefined transformations (e.g., Spark Catalyst [90], [91]), but fixed ordering can yield suboptimal plans. **Cost-Based Optimizer (CBO)** [92]–[94] estimates the cost of query execution using statistical models and employs techniques such as dynamic programming, genetic algorithms, and Monte Carlo Tree Search (MCTS) [38], [41]. **Learned optimizers (LBOs)** [12], [38], [60], [95] use ML for cost or cardinality estimation but struggle with ad-hoc AI/ML queries unseen during training.

Our Work. To close the gaps, we proposed CACTUSDB, a novel in-database AI/ML system focusing on inference queries. Like Raven [3], and different from Weld [32], Lara [53], LingoDB [89], and SDQL [54], CACTUSDB allows relational, linear-algebra, and ML operators to co-exist for expressive and flexible query processing. But our system is distinct from Raven in the following aspects: (1) Unlike Raven’s ONNX-based IR, CACTUSDB encodes AI/ML semantics using native database expressions in UDF-centric systems, forming a three-level IR. This yields tighter integration with existing optimizers, improved partial evaluation, and seamless adoption in SQL/UDF systems. (2) Instead of treating ML pipelines as monolithic graphs, CACTUSDB decomposes inference pipelines into fine-grained sub-computations, enabling selective optimizations such as factorization, push-down, and tensor-relational conversion. (3) While Raven explores a small rule-based search space (ML2SQL, ML2DNN, or none), CACTUSDB introduces a new optimizer combining query embeddings with MCTS to efficiently search the exponentially large space of co-optimization strategies.

H. Vanilla MCTS Algorithm

The Alg. 10 describes how the vanilla MCTS algorithm is used in query optimization.

I. Recommendation Queries

The three recommendation queries we used in evaluation are described in List. 1 based on the MovieLens dataset, with the corresponding ER diagram shown in Fig. 13.

In Q1, the trending model has an input size of 3, with hidden layers of 128 and 64 units. ReLU is used as the activation function. The output layer consists of a single neuron with a sigmoid function. The two tower model initialized embedding tables of dimension 32 for each categorical feature. Later, the concatenated user and movie features are then passed to a fully connected neural network with the two hidden layers, each containing 300 neurons. The output layer consists of 128

Algorithm 10: General MCTS Algorithm

Input : *root*, the state of the original query plan; *B_{iteration}*, the budget of MCTS training iterations

```

1 for i in range(Biteration) do
2   node ← root;
   /* If the current node is fully expanded,
   perform node selection. Otherwise,
   perform node expansion. */
3   while node.isTerminalState() == False do
4     if node.expanded then
5       node ← select(node);
6     else
7       node ← expand(node);
8       node' ← rollout(node);
9       break;
10  reward ← computeReward(node');
11  backpropagate(node, reward);
12 return;
```

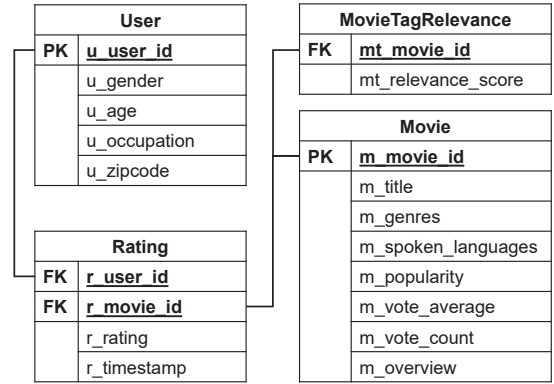


Fig. 13: ER Diagram of MovieLens Database

neurons. The input features dimensions are 129 for each user and 65 for each movie.

In Q2, the trending model is the same as the one used in Q1. The `predict_user_interest_DNN` is a fully connected neural network with an input dimension of 259 and a single hidden layer containing 128 neurons. Its output consists of two neurons with a softmax activation function. The Autoencoder model takes an input of size 140,979, representing the number of unique movie tags, with a hidden layer of 2,048 neurons. The output is a dense layer with 256 neurons. The DLRM model initializes embedding tables of dimension 128 for each categorical variable. The numerical features are processed by a one-layer MLP, a fully connected neural network with an input size of 256 and output size of 128. The categorical features’ embedding vectors are concatenated with the MLP’s output and then passed to another one-layer MLP, with an input size of 256, a hidden layer of 128 neurons, and a single-neuron output layer.

In Q3, `autoencoder` and `predict_user_interest_DNN` are both fully connected neural networks with the same architecture as described in Q2. `predict_rating_DNN` is another fully connected neural network, which has an input dimension of 5, two hidden layers with 512 and 1024 neurons, respectively, and an output layer of size 6.

```

1 // Q1
2 SELECT
3   two_tower_model (user_feature, movie_feature)
4 FROM
5   (
6     SELECT
7       concat(embedding (user_id), embedding (
8         gender), embedding (age), embedding (
9         occupation), user_avg_rating) AS
10      user_feature
11 FROM
12   USER u
13   JOIN (
14     SELECT user_id, average (rating) AS
15     user_avg_rating
16     FROM rating GROUP BY user_id
17   ) ON u.user_id=user_id
18 )
19 CROSS JOIN (
20   SELECT
21     concat(embedding (movie_id), embedding (
22     genres), movie_avg_rating) AS
23     movie_feature
24 FROM
25   movie m
26   JOIN (
27     SELECT movie_id, average (rating)
28     FROM rating GROUP BY movie_id
29   ) ON m.movie_id=movie_id
30 WHERE
31   movie.genres LIKE '%Action%' AND
32   trending_movie_DNN(movie_feature)=1
33 )
34 // Q2
35 SELECT
36   DLRM(user_feature,movie_feature,
37   movie_dense_feature)
38 FROM
39   (SELECT
40     feature_extraction1(*) AS user_feature
41   FROM
42     user
43   CROSS JOIN (
44     SELECT
45       feature_extraction2(movie_feature),
46       autoencoder(movie_relevance_tag) AS
47       movie_dense_feature
48   FROM
49     movie
50   JOIN movie_tag_feature ON movie.
51     m_movie_id=movie_tag_feature.
52     mt_movie_id
53   )
54 WHERE
55   predict_trending_movie_DNN(movie_feature
56   )=1
57   AND predict_user_interest_DNN(
58     user_feature, movie_dense_feature)
59   =1)
60 // Q3
61 SELECT
62   cosine_similarity(movie_dense_feature1,
63   movie_dense_feature2) AS relevant_score

```

```

64 FROM
65   (
66     SELECT
67       autoencoder(mt_relevance_score) AS
68       movie_dense_feature1
69   FROM
70     (
71       SELECT
72         preprocess1(user.*) AS user_feature,
73         preprocess2(movie.*) AS
74         movie_feature
75   FROM
76     user
77     CROSS JOIN movie
78     JOIN movie_tag_relevance ON mt.
79       movie_id = m_movie_id
80   )
81 WHERE
82   genres LIKE '%Fiction%'
83   AND predict_user_interest_DNN(
84     user_feature, movie_feature) = 1
85   AND predict_rating_DNN(user_feature,
86     movie_feature) > 3
87 )
88 CROSS JOIN (
89   SELECT
90     autoencoder(mt_relevance_score) AS
91     movie_dense_feature2
92 FROM
93   movie_tag_relevance
94 )

```

Listing 1: Query 1

J. Retailing-Complex Queries

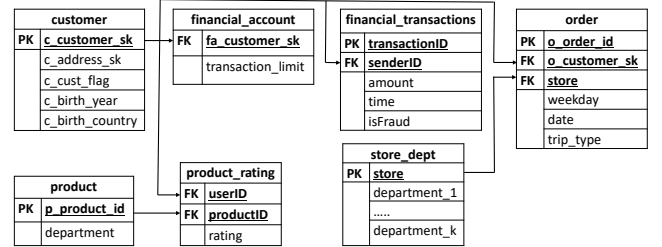


Fig. 14: Simplified ER Diagram of Retailing Dataset from TPCx-AI [103]

List. 2 presents three complex queries based on the TPCx-AI dataset, with the corresponding ER diagram shown in Fig. 14.

In Q1, `is_popular_store` is a UDF that computes a popularity score for each store based on its department availability. `trip_classifier_dnn` is a neural network comprising a customer embedding table and a fully connected neural network. The embedding table is initialized with the dimension of 16, and the resulting customer embedding vector is concatenated with additional features, forming an input of size 63. The input is then passed through a fully

connected neural network with two hidden layers of 48 and 24 neurons, respectively, followed by an output layer of size 1000, representing the trip types.

In Q2, `dnn_fraud_predict` is a deep neural network (DNN) model designed for fraud classification, consisting of an input layer of size 32, a single hidden layer with 12 neurons, and an output layer of size 2. `xgboost_fraud_predict` is an XGBoost model with 1,600 trees which take 5 features.

Q3 invokes a two-tower model that initializes multiple embedding tables for categorical variables, each with a dimension of either 16 or 8. Customer features and product features are processed separately through the user tower and product tower. The user tower is a fully connected neural network with an input size of 50, followed by two hidden layers with 128 and 64 neurons and an output layer of 16 neurons. The product tower has a similar structure but with a different input size of 25.

```

1 // Q1
2 SELECT o_order_id, trip_classifier_dnn (concat
   (order_feature, store_feature))
3 FROM (
4   SELECT o_order_id , get_order_features(
      o_customer_sk, weekday, date) AS
      order_feature, store_feature
5   FROM order o
6   JOIN (
7     SELECT store, concat (*) AS store_feature
8     FROM store_dept_wide s
9   ) ON o. store = s.store
10  WHERE o. weekday != 'Sunday' AND
      is_popular_store (store_feature) = 1
11 )
12
13
14 // Q2
15 SELECT transactionID
16 FROM (
17   SELECT c_customer_sk , get_customer_feature
      (c_address_sk, c_cust_flag, c_birth_year
      , c_birth_country, transaction_limit) AS
      customer_feature
18   FROM customer
19   JOIN (
20     SELECT fa_customer_sk, transaction_limit
21     FROM financial_account
22   ) ON c_customer_sk = fa_customer_sk
23   WHERE c_cust_flag = 0
24   JOIN (
25     SELECT transactionID, senderID,
      get_transaction_feature (*) AS
      transaction_feature
26     FROM financial_transactions t
27   ) ON c_customer_sk = senderID
28   WHERE age_during_transaction ( c_birth_year
      , t.time) >= 18 AND is_working_day (t.
      time) = 1
29 )
30 WHERE xgboost_fraud_predict(concat (
   customer_feature, transaction_feature)) >=
   0.5 AND dnn_fraud_predict(concat (
   customer_feature, transaction_feature)) =
   1
31

```

```

32
33
34 // Q3
35 SELECT two_tower_model (customer_feature,
   product_feature)
36 FROM (
37   SELECT concat(embedding (c_customer_sk),
      embedding (c_address_sk), c_cust_flag,
      embedding (get_age (c_birth_year)),
      embedding (birth_country),
      customer_avg_rating) AS customer_feature
38   FROM customer c
39   JOIN (
40     SELECT userID, AVG (rating) AS
      customer_avg_rating
41     FROM product_rating GROUP BY userID
42   ) AS cr ON c. c_customer_sk = cr.userID
43 ) AS customer_outer
44 CROSS JOIN (
45   SELECT concat(embedding (p_product_id),
      embedding(department),
      product_avg_rating) AS product_feature
46   FROM product p
47   JOIN (
48     SELECT productID, AVG (rating) AS
      prod_avg_rating
49     FROM product_rating GROUP BY productID
50   ) AS pr ON p. p_product_id = pr.productID
51   WHERE pr.prod_avg_rating >= 4.0
52 ) AS prod_outer

```

Listing 2: Queries on Retailing Dataset

K. LLM Queries

List. 3 describes the two LLM queries used in our evaluation. We utilize OpenAI’s RESTful API to invoke the gpt-3.5-turbo model to summarize the user/movie information and make the recommendations. The `trending_movie_classifier` is a fully connected neural network with the same structure as the one described in Sec. 4. Q2 employs a RAG-based approach to retrieve movies’ relevant information by performing a nearest-neighbor vector search on pre-built document embedding vectors using Faiss’s cosine similarity index.

```

1 // Q1: Using LLM for Movie Recommendation
2 SELECT LLM('Please give a recommendation score
   and explain why', LLM('Please summarize',
   u.description), LLM('Please summarize', m
   .description))
3 FROM user u CROSS JOIN movie m
4 WHERE m.spoken_language LIKE % English % AND
   trending_movie_classifier(m.popularity, m.
   vote_average, m.vote_num) = True;
5
6 // Q2: Using RAG to retrieve relevant
   information with given movie title and
   using LLM for recommendation
7 SELECT LLM('Please give a recommendation score
   and explain why', LLM('Please summarize',
   u.description), RAG(m.title))
8 FROM user u CROSS JOIN movie m
9 WHERE m.spoken_language LIKE % English % AND
   trending_movie_classifier(m.popularity, m.
   vote_average, m.vote_num) = True;

```


Listing 3: LLM Queries

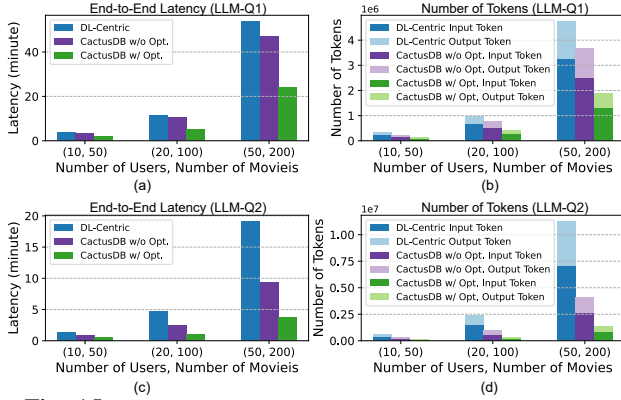


Fig. 15: Evaluating the LLM Queries with R4-1 and R1-3

LLM Inference Queries. In this benchmark, the overheads of LLM inferences dominate the end-to-end latency. In addition, the monetary costs of LLM usage linearly increase with the number of tokens sent to and received from LLM. Q2 replaces the LLM-based movie summarization with RAG, which results in faster execution but increases the number of tokens compared to Q1. As shown in Fig. 15,

our CACTUSDB is capable of factorizing the nested LLM functions and pushing down the LLM summarization call and RAG call before the `cross_join` to avoid repeatedly LLM inferences on the same records, resulting in $3\times$ speed up and 72.4% reduction of tokens (and costs) on average. We noticed that even without enabling our reusable MCTS query optimizer, CACTUSDB benefits from Velox’s built-in runtime optimization, such as filter push-down. These runtime optimizations effectively reduced the total number of input/output tokens sent to the LLMs by 25% for Q1 and 59% for Q2 on average with end-to-end latency speedups of $1.1\times$ to $2.1\times$.

L. Comparison of Optimization Algorithms on Recommendation Queries

TABLE IV: Comparison of Optimization Algorithms (unit: second)

	Recommendation-Q1			Recommendation-Q2			Recommendation-Q3		
	Opt.	Exec.	Total	Opt.	Exec.	Total	Opt.	Exec.	Total
Un-optimized	NA	144.2	144.2	NA	13.6	13.6	NA	36.5	36.5
Arbitrary	0.9	1.8	2.7	5.1	131.3	136.3	4.4	23.7	28.1
Heuristic	0.2	5.2	5.4	0.3	12.3	12.6	0.1	16.3	16.3
Vanilla MCTS	5.2	0.3	5.5	6.8	6.8	13.6	10.3	8.7	18.9
Reusable MCTS-two models	1.5	0.2	1.7	2.1	6.5	8.6	3.6	8.8	12.4

Recommendation queries. We break down the end-to-end latency of each query into two parts: query optimization latency and query execution latency. Tab. IV compares the latency for query optimization (Opt.), query execution (Exec.), and the total of them. **Our reusable MCTS strikes the best trade-off between overhead and effectiveness compared to other baselines.** Even though it introduces additional overhead for query embedding, the query optimization latency is greatly

saved by reusing the MCTS. The arbitrary optimization strategy could lead to better or worse query execution time compared to unoptimized queries. That is because not all optimization rules will be beneficial. Heuristic optimization strategy will always lead to better efficiency (optimization latency). However, simply relying on pre-defined heuristics cannot exploit the best optimization techniques for all queries. Vanilla MCTS demonstrates effectiveness in searching for optimal query plans due to its query-specific optimization. However, it requires building an MCTS search tree from scratch every time, which worsens the optimization latency.

M. Synthetic Model Sampling

We design a model sampling strategy to generate diverse architectures for collecting the training data for the Model2Vec model. The strategy is built on a set of reusable model-building blocks, which we denote as parameterized operators:

- ConvolutionalLayer (in, out, kernel, stride, padding)
- PoolingLayer (kernel, stride, padding)
- LinearLayer (in, out, activation, bias)
- BatchNormalization (num_features, ϵ)
- Dropout(p)
- EmbeddingLayer (num_tuples, dim)
- SVD (dim)
- Decision_Forest (n_{trees} , d_{max})
- DecisionTree (d_{max})
- Activation $\in \{\text{ReLU}, \text{Sigmoid}, \text{Tanh}, \text{Softmax}\}$
- Concat ($\cdot$)
- CosineSimilarity ($\cdot, \cdot$)
- Flatten ($\cdot$)

Each operator can be parameterized within a configurable range, and blocks may be repeated between a minimum and maximum depth. Using these primitives, users can compose customized model templates based on their needs. In our work, we define a set of representative model templates, which are used to generate training data, as illustrated in List. 4:

```

1 // Multi-Layer Perceptron (MLP)
2 MLP :=
3   Repeat{min_repeat, max_repeat}{
4     LinearLayer(RANGE(in_min, in_max),
5                 RANGE(out_min, out_max),
6                 CHOICE(Activation))
7   }
8
9 // Two-Tower Model
10 Tower :=
11   Repeat{min_repeat, max_repeat}{
12     LinearLayer(RANGE(in_min, in_max),
13                 RANGE(out_min, out_max),
14                 CHOICE(Activation))
15   }
16 TwoTowerModel := CosineSimilarity(Tower, Tower)
17
18 // Deep Learning Recommendation Model (DLRM-style)
19 DLRM :=
20   Concat (

```

```

21 Repeat{min_repeat, max_repeat}{
22     EmbeddingLayer(RANGE(num_min, num_max),
23                     RANGE(dim_min, dim_max))
24 },
25 Repeat{min_repeat, max_repeat}{
26     LinearLayer(RANGE(in_min, in_max),
27                 RANGE(out_min, out_max),
28                 CHOICE(Activation))
29 }
30 )
31 -> Repeat{min_repeat, max_repeat}{
32     LinearLayer(RANGE(in_min, in_max),
33                 RANGE(out_min, out_max),
34                 CHOICE(Activation))
35 }
36 // Convolutional Neural Network (CNN)
37 CNN :=
38 Repeat{min_repeat, max_repeat}{
39     ConvolutionalLayer(RANGE(in_min, in_max),
40                        RANGE(out_min, out_max)
41                        ,
42                        RANGE(kernel_min,
43                              kernel_max))
43     -> BatchNorm(num_features)
44     -> CHOICE(Activation)
45     -> PoolingLayer(RANGE(kernel_min,
46                             kernel_max))
46 }
47 -> Flatten()
48 -> Repeat{min_repeat, max_repeat}{
49     LinearLayer(RANGE(in_min, in_max),
50                 RANGE(out_min, out_max),
51                 CHOICE(Activation))
52 }
53 // Decision Forest
54 DecisionForest :=
55 DecisionForest(RANGE(n_trees_min,
56                      n_trees_max),
57                RANGE(depth_min, depth_max))

```

Listing 4: Predefined model templates

In practice, when sampling a model, the system randomly selects one of these templates and instantiates its building blocks by drawing parameter values (e.g., layer widths, kernel sizes, embedding dimensions, number of trees) from the specified ranges. This process yields a rich variety of architectures, while ensuring that the sampled models remain consistent with patterns observed in real-world in-database ML workloads.

N. Synthetic Query Templates

We describe the query templates used to generate training data and evaluation queries. In the MovieLens dataset, we convert the three queries described in Sec. 4 into templates and pick the seven most voted data science pipelines from Kaggle [5]. In the TPCx-AI [45] dataset, in addition to the tree queries we described in Sec. 4, we convert the other use cases queries into templates as illustrated in List. 5 and List. 6. The query sampler randomly samples a model architecture and configuration parameters, and randomly samples filters with different selectivity.

```

1 // Template 4: User Rating Prediction for
  Movies
2 SELECT DNN(process_gender(gender),
  normalize_age(age), normalize_occupation(
  occupation), split_genres(genres))
3 FROM user CROSS JOIN movie
4 [WHERE AGE_FILTER | GENDER_FILTER |
  OCCUPATION_FILTER | GENRE_FILTER];
5
6 // Template 5: User Opinion Prediction (
  Negative/Neutral/Positive)
7 SELECT DNN(user_id, movie_id)
8 FROM user
9 [WHERE AGE_FILTER | GENDER_FILTER |
  OCCUPATION_FILTER | ZIP_CODE_FILTER];
10
11 // Template 6: Recommend Movies to Users via
  SVD
12 SELECT SVD(user_id, movie_id)
13 FROM user CROSS JOIN movie
14 [WHERE AGE_FILTER | GENDER_FILTER |
  OCCUPATION_FILTER | ZIP_CODE_FILTER |
  GENRE_FILTER];
15
16 // Template 7: Rating Prediction with
  Collaborative Filtering
17 SELECT LightFM(user_id, movie_id)
18 FROM user CROSS JOIN movie
19 [WHERE AGE_FILTER | GENDER_FILTER |
  OCCUPATION_FILTER | ZIP_CODE_FILTER |
  GENRE_FILTER];
20
21 // Template 8: Rating Prediction with
  AutoEncoder
22 SELECT AutoEncoder(user_id, movie_id)
23 FROM user
24 [WHERE AGE_FILTER | GENDER_FILTER |
  OCCUPATION_FILTER | ZIP_CODE_FILTER];
25
26 // Template 9: Detect Gender Stereotypes with
  DNN
27 SELECT DNN(prepare_feature(...))
28 FROM ratings JOIN movie ON ratings.movie_id =
  movie.movie_id
29 [WHERE GENRE_FILTER | TIMESTAMP_FILTER];
30
31 // Template 10: Rating Prediction
32 SELECT DNN(movie_id, age, occupation)
33 FROM user CROSS JOIN movie
34 [WHERE AGE_FILTER | GENDER_FILTER |
  OCCUPATION_FILTER | ZIP_CODE_FILTER |
  GENRE_FILTER];

```

Listing 5: Query templates for MovieLens dataset

```

1 // Template 4: Product Rating Prediction with
  SVD
2 SELECT SVD(user_id, product_id)
3 FROM product_rating
4 JOIN product ON productID = p_product_id
5 JOIN customer ON c_customer_sk = userID
6 [WHERE PRODUCT_DEPARTMENT_FILTER |
  CUSTOMER_BIRTH_DATE_FILTER |
  CUSTOMER_BIRTH_COUNTRY_FILTER];
7
8 // Template 5: Spam Review Detection with DNN

```

```

9 SELECT DNN(tf_idf_features(text))
10 FROM review_comments
11 [WHERE ID_FILTER];
12
13 // Template 6: Classification of Trips with
14   DNN/Decision Forest
15 SELECT DNN/DecisionForest(prepare_feature(...)
16   )
17 FROM (
18   SELECT o_order_id, department, quantity,
19     SUM(quantity) AS scan_count,
20     MIN(EXTRACT(DOW FROM date)) AS
21       weekday
22 FROM tpcxai_order_serving
23 JOIN tpcxai_lineitem_serving ON o_order_id =
24   li_order_id
25 JOIN tpcxai_product_serving ON li_product_id
26   = p_product_id
27 GROUP BY o_order_id, date, department,
28   quantity
29 )
30 [WHERE TIMESTAMP_FILTER | DEPARTMENT_FILTER |
31   WEEKDAY_FILTER | PRICE_FILTER |
32   QUANTITY_FILTER];
33
34 // Template 7: Fraud Detection
35 SELECT DNN/LogisticRegression(concat(EXTRACT(
36   HOUR FROM time)/23, amount/
37   transaction_limit))
38 FROM financial_transactions
39 JOIN financial_account ON sender_id =
40   fa_customer_sk
41 JOIN customer ON fa_customer_sk =
42   c_customer_sk
43 [WHERE TIMESTAMP_FILTER | AMOUNT_FILTER |
44   CUSTOMER_BIRTH_INFO_FILTER];
45
46 // Template 8: Sales Prediction
47 SELECT DNN(store_id_encoder(store_id),
48   department_encoder(department),
49   normalize_week(week))
50 FROM store_dept
51 [WHERE DEPARTMENT_FILTER | WEEK_FILTER |
52   STORE_FILTER];
53
54 // Template 9: Customer Segmentation
55 SELECT KMeans/DNN(prepare_feature(...))
56 FROM (
57   SELECT YEAR(date) AS invoice_year,
58     quantity * price AS row_price,
59     or_return_quantity * price AS
60       return_row_price
61 FROM order
62 JOIN lineitem ON o_order_id = li_order_id
63 JOIN order_returns ON or_order_id =
64   o_order_id
65 JOIN product ON p_product_id = li_product_id
66 )
67 [WHERE DATE_FILTER | STORE_FILTER |
68   PRODUCT_FILTER | DEPARTMENT_FILTER];
69
70 // Template 10: Customer Satisfaction
71   Prediction
72 SELECT DNN(prepare_feature(...))
73 FROM customer
74 CROSS JOIN product
75 JOIN lineitem ON li_product_id = p_product_id

```

```

36 [WHERE CUSTOMER_INFO_FILTER |
   DEPARTMENT_FILTER | PRICE_FILTER];

```

Listing 6: Query templates For TPCx-AI dataset